RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS1006
Data Structures

Unit – I Abstract Data Types
&
Memory Allocation

Course Outcomes

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

After completing the course, the students will be able to

CO1: Understand and explore the concepts of Linear and Non-Linear Data Structures to develop solutions for diverse problems.

CO2: Apply the concepts of Hashing, Sorting and Searching techniques to get an Optimized solution.

CO3: Evaluate and apply appropriate data structure(s) for real-world problems.

CO4: Demonstrate good coding practices involving lifelong learning.

Syllabus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Module – 1 Abstract Data Types & Memory Allocation **9 Hours**

Strategies for choosing the appropriate data structure. Abstract data types. Stack Memory - Compile Time, Static memory allocation. Heap Memory - Runtime, dynamic memory allocation. Arrays – Revision of basic operations. Implementation of basic operations – Create, Insert, Traverse, Delete, Search and Update.

Module – 2 Linked List **9 Hours**

Linked lists – Types – Singly Linked List – Doubly Linked List – Circular Linked List -(Circular Singly, Circular Doubly). Implementation of operations – Create, Insert, Traverse, Delete, Search and Update.

Module – 3 Stacks and Queues **9 Hours**

Introduction to Stacks and Queues. Implementation of their basic operations. Use basic arrays & link list to create stack & queue data structures.

Syllabus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Module – 4 Sorting, Searching, and Hashing **9 Hours**

Linear Search, Binary Search, Bubble Sort, Selection Sort, Insertion Sort. Hashing – Hash tables, has functions, strategies to avoid and resolve collisions.

Module – 5 Trees **9 Hours**

Binary Tree, Binary Search Tree, AVL Tree, B-Trees, Common operations on these trees such as select min, select max, create, insert, delete. Perform various types of Tree Traversals.

Data Structure – Basics

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Data structures are generally based on the ability of a computer to fetch and store data at any place in its memory, specified by a pointer.
- Thus, the array and record data structures are based on computing the addresses of data items with arithmetic operations

Data Structures + Algorithms = Program

- Program -> A Set of Instructions
- Data Structure -> A Container stores Data
- Algorithm -> Logic + Control

Data Structure – Basics

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

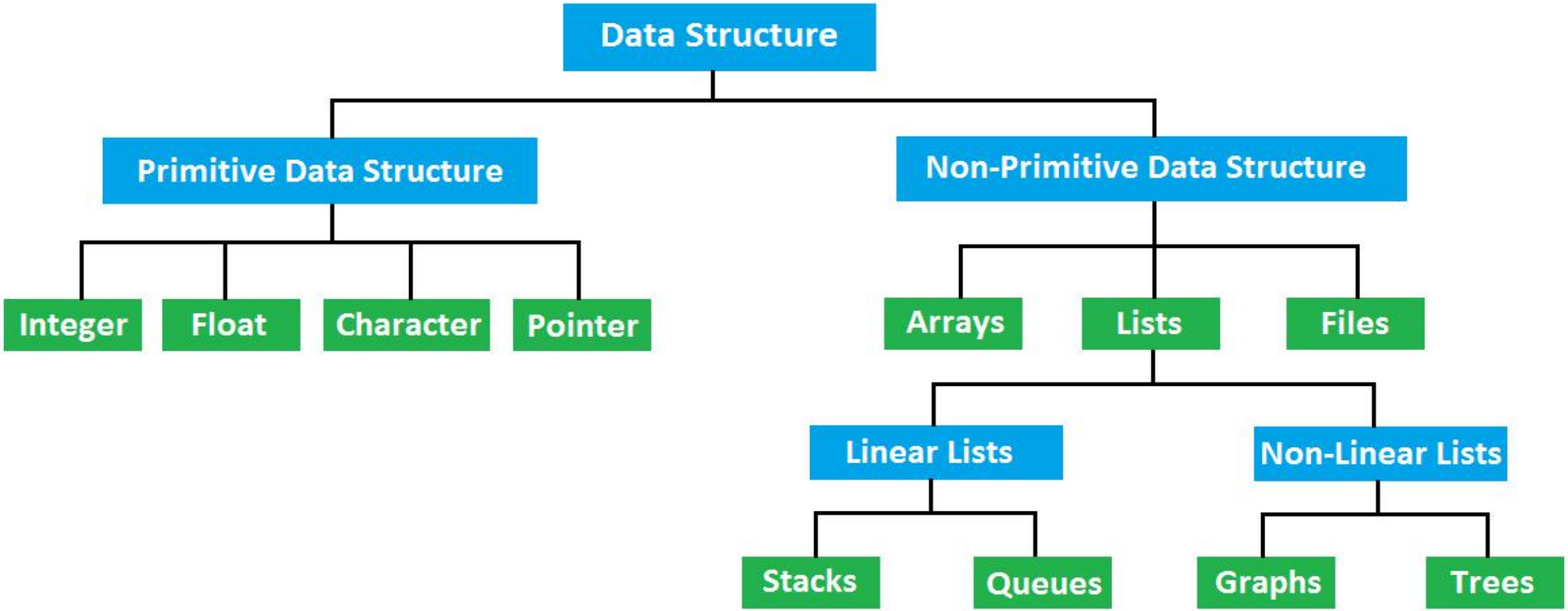
- Data: simply a value or set of values of different types which is called data types like string, integer, char etc.
- Structure: Way of organizing information, so that it is easier to use.

Data Structure ..

- A data structure is a particular way of organizing data in a computer so that it can be used efficiently.

- A scheme for organizing related pieces of information.

Data Structures – Classification



Definitions

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Primitive Data Structure:

✔ A Primitive Data Structure used to represent the standard data types of any one of the computer languages (integer, Character, float etc.).

Non - Primitive Data Structure :

✔ A Non - Primitive Data Structure can be constructed with the help of any one of the primitive data structure.

- It can be structured having a specific functionality.
- It can be designed by user.
- It can be classified as Linear and Non-Linear Data Structure.

Definitions

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Linear Data Structures:

- A linear data structure traverses the data elements sequentially, in which only one data element can directly be reached.
- Ex: Arrays, Linked Lists ,Stacks, Queues

Non-Linear Data Structures:

- Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.
- Ex: Trees, Graphs ,Heaps

Basic Operations on Data structures

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- The basic operations that are performed on data structures are as follows:

1. **Traversal:** Traversal of a data structure means processing all the data elements present in it exactly once.

2. **Insertion:** Insertion means addition of a new data element in a data structure.

3. **Deletion:** Deletion means removal of a data element from a data structure if it is found.

Basic Operations on Data structures

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

4. Searching: Searching involves searching for the specified data element in a data structure.

5. Sorting: Arranging data elements of a data structure in a specified order is called sorting.

6. Merging: Combining elements of two similar data structures to form a new data structure of the same type, is called merging.

Abstract Data Type (ADT)

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- An Abstract Data type refers to set of data values and associated operations that are specified accurately, independent of any particular implementation. (Or)
- ADT is a user defined data type which encapsulates a range of data values and their functions. (Or)
- An **Abstract Data Type** is a mathematical model of a **data structure**. It describes a container which holds a finite number of objects where the objects may be associated through a given binary relationship.
- Ex: List, Stack ,Queue

The List ADT

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- ✔ A **list** or **sequence** is an abstract data type that represents a countable number of ordered values, where the same value may occur more than once.
- ❖ A sequence of zero or more elements $A_1, A_2, A_3, \ldots A_N$
- ❖ length of the list A_1: first element A_N: last element A_i: position i
- ❖ If N=0, then empty list
- ❖ Linearly ordered -> A_i precedes A_{i+1} (or) A_i follows A_{i-1}
- ✔ Lists are a basic example of containers, as they contain other values. If the same value occurs multiple times, each occurrence is considered a distinct item.

The List ADT – Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

✔A list contains elements of same type arranged in sequential order and following operations can be performed on the list.

- get() – Return an element from the list at any given position.
- insert() – Insert an element at any position of the list.
- remove() – Remove the first occurrence of any element from a non-empty list.

The List ADT – Operations

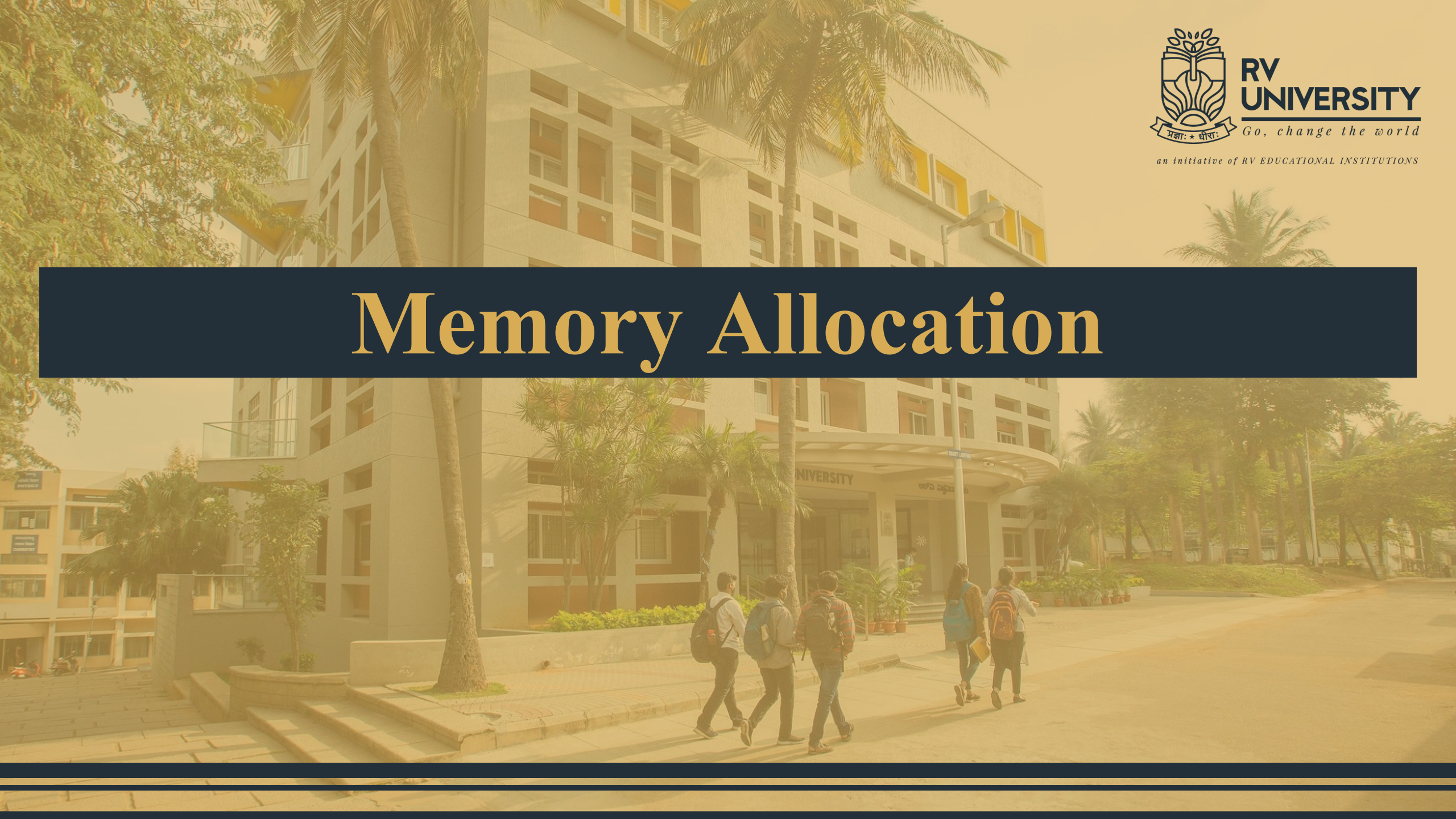
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- removeAt() – Remove the element at a specified location from a non-empty list.
- replace() – Replace an element at any position by another element.
- size() – Return the number of elements in the list.
- isEmpty() – Return true if the list is empty, otherwise return false.
- isFull() – Return true if the list is full, otherwise return false.

Implementation of List ADT

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Each operation associated with the ADT is implemented by one or more subroutines.
- Two standard implementations for the list ADT
  - ✔ Array-based
  - ✔ Linked list
- An array is a *random access* data structure, where each element can be accessed directly and in constant time.
  - A typical illustration of random access is a book - each page of the book can be open independently of others.



RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Memory Allocation

Syllabus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Unit 1: Array and Linked List

~~12 hours~~

- Strategies for choosing the appropriate data structure,
- Abstract Data types,
- **Stack Memory – Compile time,**
- **Static Memory Allocation,**
- **Heap Memory – Runtime ,**
- **Dynamic Memory Allocation**.
- Arrays
- Revision of Basic Operations
- ~~Linked Lists (Singly, Doubly, Circular Singly, Circular Doubly).~~
- ~~Implementation of Operations – Create, Insert, Traverse, Delete, Search and Update.~~

Memory Layout in 'C' Program

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

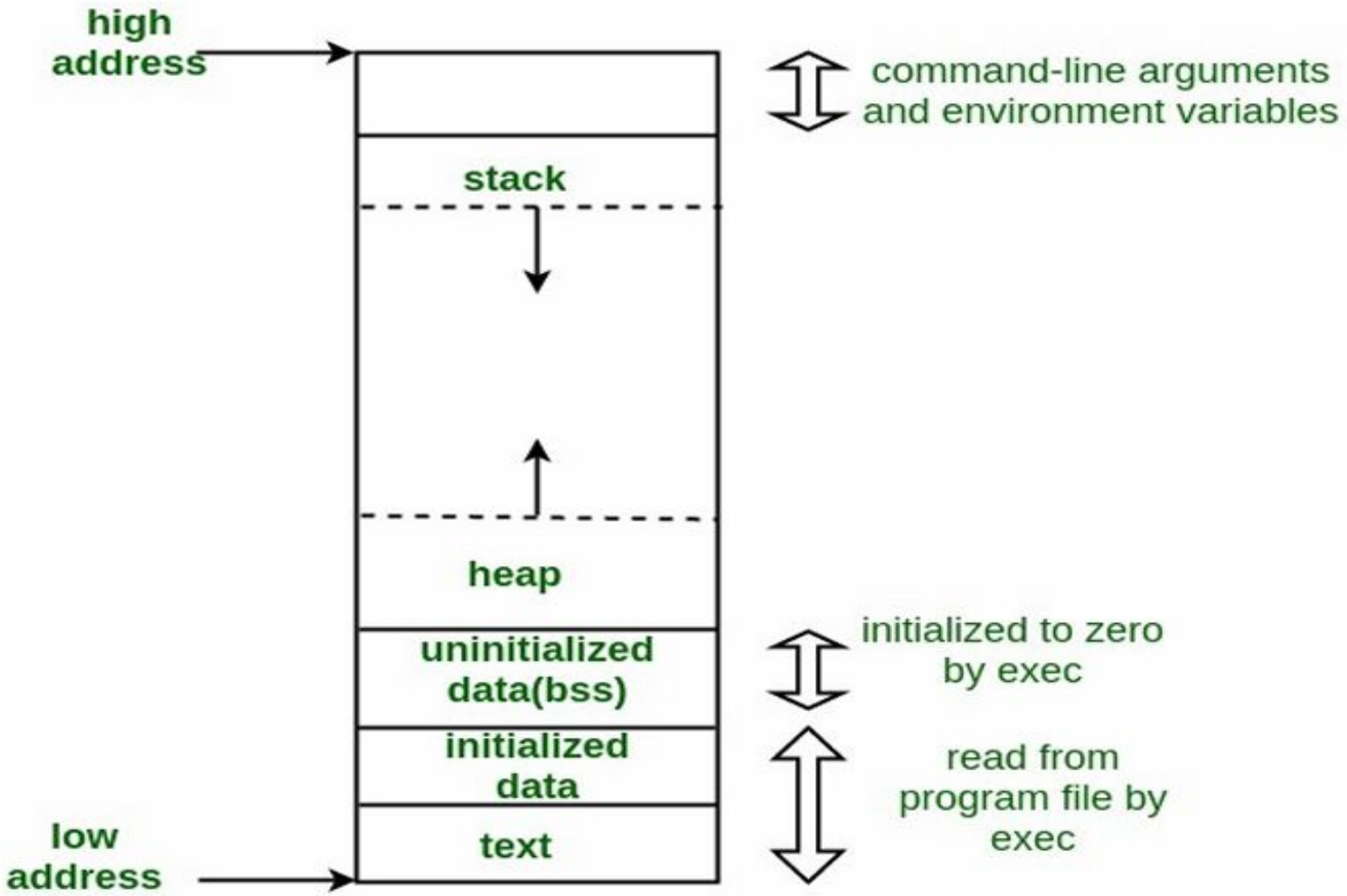
- After compiling a C program, a binary executable file(.exe) is created, and when we execute the program, this binary file loads into RAM in an organized manner.

- After being loaded into the RAM, memory layout in C Program has six components which are **text segment**, **initialized data segment, uninitialized data segment, command-line arguments, stack, and heap**.

- Each of these six different segments stores different parts of code and have their own read, write permissions.

- If a program tries to access the value stored in any segment differently than it is supposed to, it results in a **segmentation fault error**.

Memory Layout in 'C' Program



Text Segment

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- After we compile the program, a binary file generates, which is used to execute our program by loading it into RAM. This binary file contains instructions, and these instructions get stored in the text segment of the memory.

- Text segment has read-only permission that prevents the program from accidental modifications.

- Text segment in RAM is shareable so that a single copy is required in the memory for frequent applications like a text editor, shells, etc.

Initialized Data Segment

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Initialized data segment or data segment is part of the computer's virtual memory space of a C program that contains values of all external, global, static, and constant variables whose values are initialized at the time of variable declaration in the program.
- Because the values of variables can change during program execution, this memory segment has read-write permission.
- We can further classify the data segment into the read-write and read-only areas.
- *const* variable comes under the read-only area

Program

```c
#include<stdio.h>

/* global variables stored in the read-write part of
   initialized data segment
 */
int global_var = 50;
char* hello = "Hello World";
/* global variables stored in the read-only part of
   initialized data segment
 */
const int global_var2 = 30;

int main() {
    // static variable stored in initialized data segment
    static int a = 10;
    // ...
    return 0;
}
```

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Uninitialized Data Segment

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- An uninitialized data segment is also known as bss (**b**lock **s**tarted by **s**ymbol).

- The program loaded allocates memory for this segment when it loads. Every data in bss is initialized to arithmetic 0 and pointers to null pointer by the kernel before the C program executes.

- BSS also contains all the static and global variables, initialized with arithmetic 0.

- Because values of variables stored in bss can be changed, this data segment has read-write permissions

Practice Questions

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Write a program to initialize four pointers of primitive data type (char, integer, float, double).

  Use ‘sizeof’ operator and print the amount of memory occupied by a pointer to a primitive data type (char, integer, float, double).

- Write a program to initialize four arrays of primitive data type (char, integer, float, double).

  Hint: You must create an integer array, char array, float array and double array of size 5.

  Use pointer to an array for each of the four arrays and,

  Use ‘sizeof’ operator and print the amount of memory occupied by a pointer to an array holding a primitive data type (char, integer, float, double).

  Use ‘sizeof’ operator and print the amount of memory occupied by an array holding primitive data types (char, integer, float, double)

Stack Memory

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- The stack memory makes use of the stack data structure.

- Stack memory is a memory usage mechanism that allows the system memory to be used as temporary data storage that behaves as a LIFO (Last In First Out) buffer.

- Stack segment stores the value of local variables and values of parameters passed to a function (by creating stack frame for each function) along with some additional information like the instruction's return address, which is to be executed after a function call.

- Stack grows in the direction opposite to heap.

Program


```c
#include<stdio.h>
int total;
int Square(int x)
{
    return x*x;
}
int SquareOfSum(int x,int y)
{
    int z = Square(x+y);
    return z;
}
int main()
{
    int a = 4, b = 8;
    total = SquareOfSum(a,b);
    printf("output = %d",total);
}
```

- One of the common cases of stack overflow is when you write bas recursive programs. Your program may go into infinite loop.
- Memory set aside for stack doesn't grow during run time.
- Application cannot request more memory from stack.
- Allocation and deallocation of memory happens using a set of rules.
- It is not possible to manipulate the scope of the variable if it is on stack.

Heap Memory

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Unlike stack, applications **heap memory is not fixed**.
- It can change during run time.
- Heap is used for memory which is **allocated during the run time** (dynamically allocated memory).
- Heap is referred to as dynamic memory and using heap for program/application is referred to as **dynamic memory allocation**.
- Heap is also called as the **free pool of memory**.
- To use dynamic memory in C we need to know about four functions: **malloc(), calloc(), free(), realloc()**.These four functions are used to manage allocations in the heap segment

Types of Memory Allocation in 'C'

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

C programming or any other programming language basically support two types of memory allocation.

1. Compile time memory allocation

2. Runtime memory allocation

Compile Time Memory Allocation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Compile time memory allocation means reserving memory for variables, constants **during the compilation process**.
- The memory allocated is **fixed and cannot be increased or decreased** during run time.
- So, you must exactly know how many bytes you require?
- Many text also refer compile time memory allocation as **static or stack memory allocation**.
- In static memory allocation, once the memory is allocated, the memory size can not change

Key Features of Static Memory Allocation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Allocation and deallocation are **done by the compiler**.
- It **uses** a data structures **stack** for static memory allocation.
- Variables get **allocated permanently**.
- **No reusability**.
- Execution is **faster** than dynamic memory allocation.
- Memory is allocated before runtime.
- It is **less efficient**

Run Time Memory Allocation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Memory allocation done at the time of execution(run time) is known as dynamic memory allocation.

- **Heap** is referred to as dynamic memory. Making use of heap for allocation of memory is known as dynamic memory allocation.

NOTE: Not be to confused with heap data structure.

- Also sometimes known as free pool of memory.

Run Time Memory Allocation

- **Pointers play an important role** in Dynamic Memory Allocation.
- Only way to access memory on heap is through pointers.
- To use dynamic memory in C we need four built–in functions:

✔ malloc()

✔ calloc()

✔ realloc()

✔ free()

Key Features

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- We can also **reallocate memory size** if needed.
- Dynamic Allocation is **done at run time**.
- **No memory wastage**

malloc()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- The “malloc” or “memory allocation” method in C is used to dynamically allocate a **single large block of memory** with the specified size.
- It **returns a pointer of type void** which can be cast into a pointer of any form.
- It initializes **each block** with the **default garbage value** initially.

Syntax: ptr = (cast-type*) malloc(byte-size)

malloc() – Example

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Case 1.

```
ptr = (int*) malloc(1 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 4 bytes of memory.

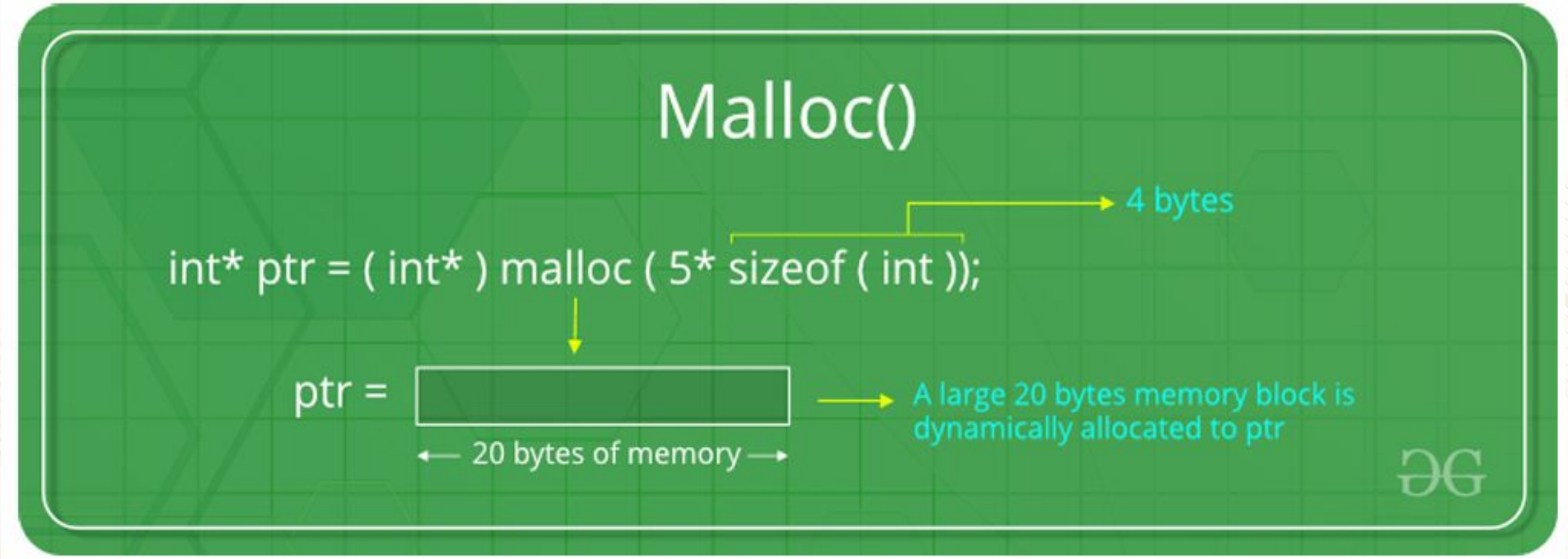
Case 2.

```
ptr = (int*) malloc(100 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And the pointer ptr holds the address of the first byte in the allocated memory

malloc()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS



calloc()

"calloc" or "contiguous allocation" method in C is used to dynamically allocate the **specified number of blocks** of memory of the specified type.

- It is very similar to malloc() but has two different points and these are:

1. It **initializes each block with** a default value '**0**'.
2. It has two parameters or arguments as compared to malloc().

Syntax: ptr = (cast-type*)calloc(n, element-size);

'n' is the no. of elements and element-size is the size of each element.

calloc() – Example

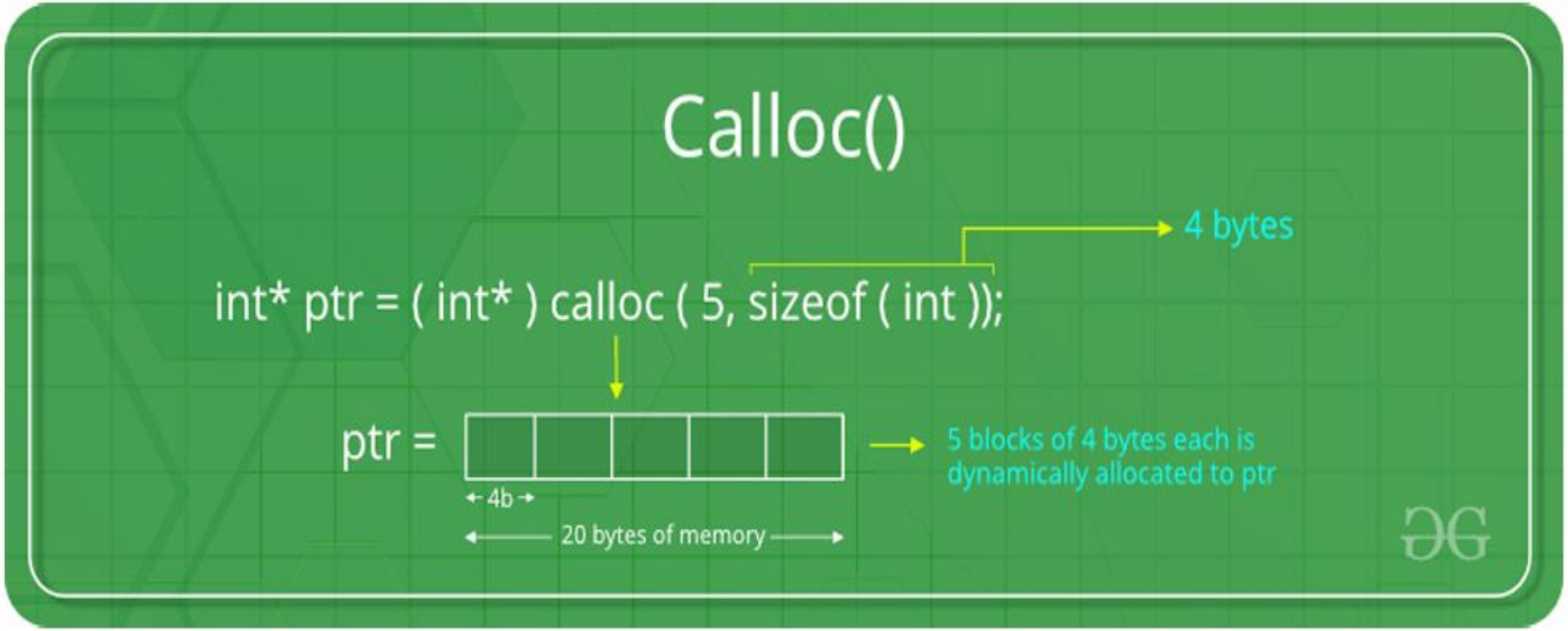
For Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

This statement allocates contiguous space in memory for 25 elements each with the size of the float

calloc()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS



realloc()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

•“realloc” or “re-allocation” method in C is used to dynamically change the memory allocation of a previously allocated memory.

• In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**.

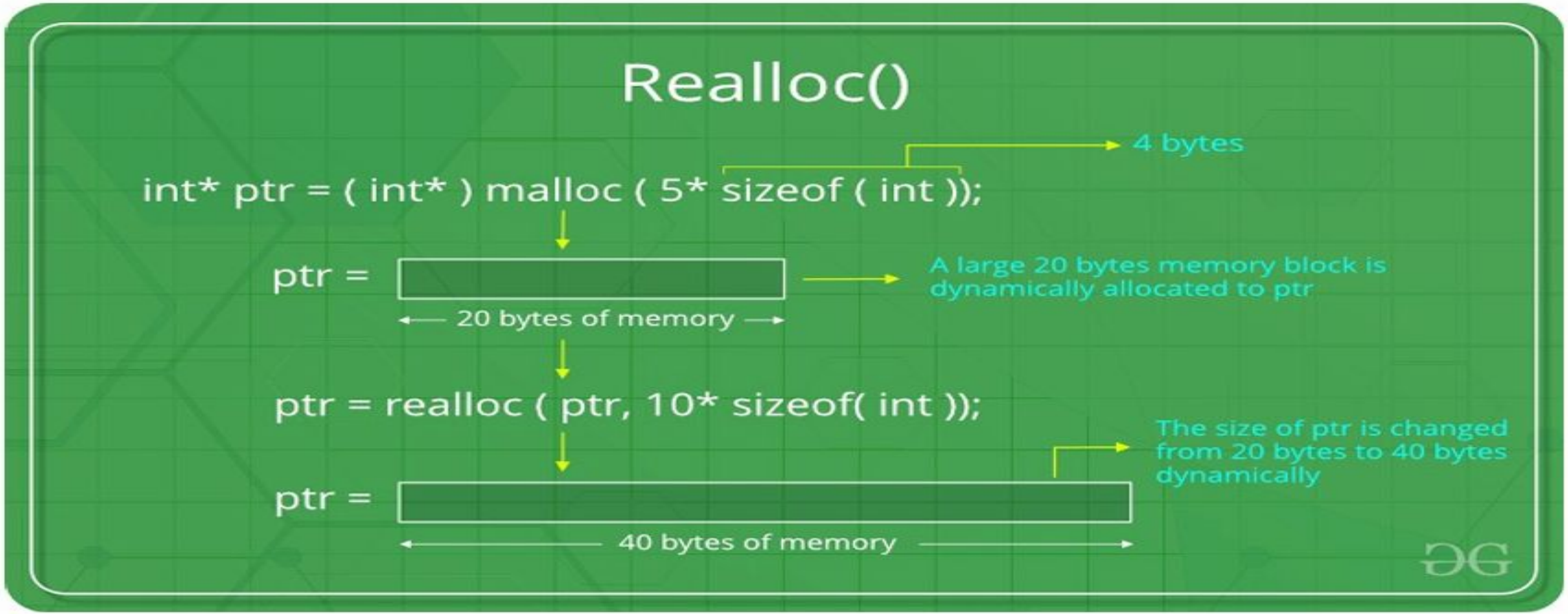
• re-allocation of memory maintains the already present value and **new blocks will be initialized with the default garbage value**.

Syntax: ptr = realloc(ptr, newSize);

• where ptr is reallocated with new size 'newSize'

realloc()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS



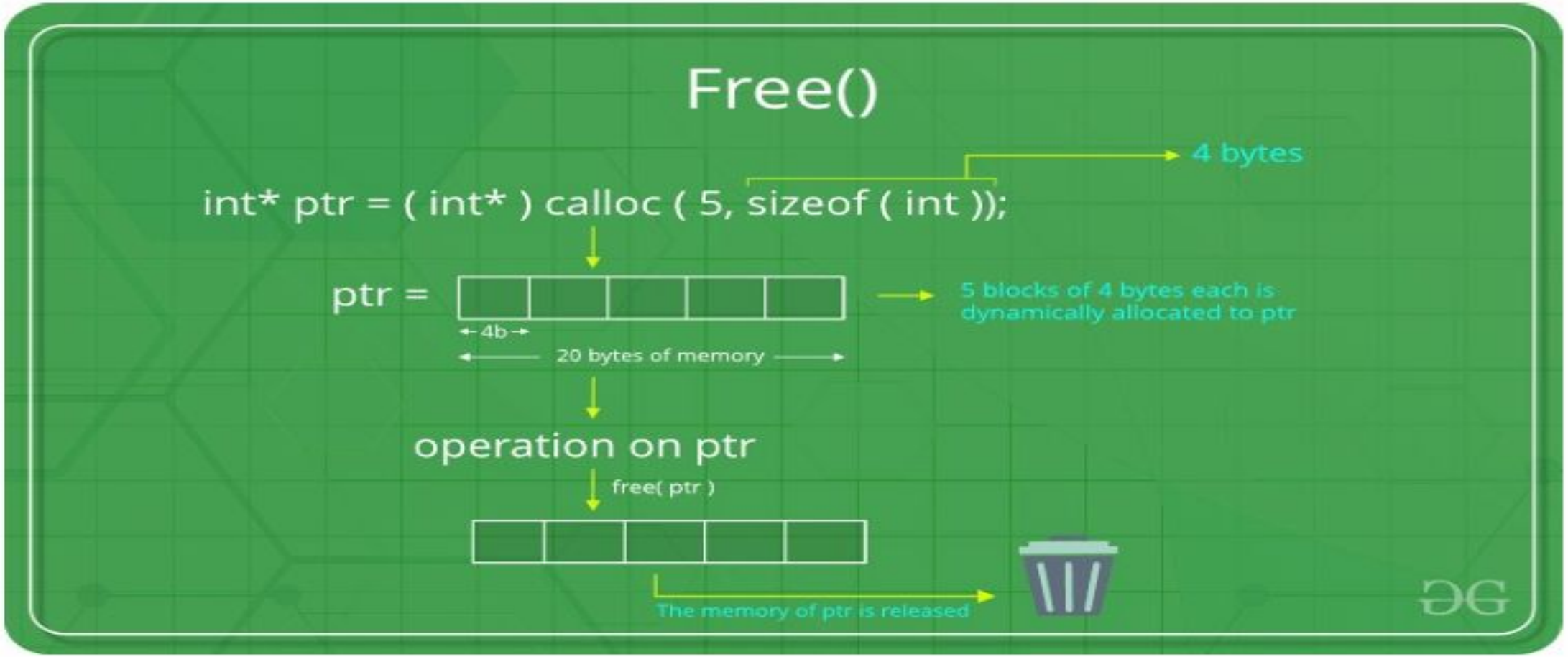
free()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- "free" method in C is used to dynamically de-allocate the memory.
- The memory allocated using functions malloc() and calloc() is not de allocated on their own.
- Hence the free() method is used, whenever the dynamic memory allocation takes place.
- It helps to reduce wastage of memory by freeing it.

Syntax: free(ptr);

free()


Free()
4 bytes
int* ptr = (int*) calloc (5, sizeof (int));
ptr =
4b
20 bytes of memory
5 blocks of 4 bytes each is dynamically allocated to ptr
operation on ptr
free(ptr)
The memory of ptr is released

Comparision

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Compile Time Memory Allocation	Run Time Memory Allocation
Memory is allocated during program compilation	Memory is allocated during runtime
Cannot reuse allocated memory	Can reuse allocated memory after releasing memory using free() function
Library functions are not required to allocate memory	Requires library functions like malloc(), calloc(), realloc() to allocate memory at runtime.
Cannot modify size of allocated memory	Can modify size of allocated memory using realloc()

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Arrays & its operations

Syllabus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Unit I

9 hours

- Strategies for choosing the appropriate data structure,
- Abstract Data types,
- Stack Memory – Compile time,
- Static Memory Allocation,
- Heap Memory – Runtime ,
- Dynamic Memory Allocation.
- **Arrays**
- **Revision of Basic Operations**

Array Based Implementation of List

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- An Array is a data structure which can store a fixed-size sequential collection of elements of the same type.
- Array declaration use numbers[0], numbers[1],..., numbers[99] to represent individual variables.
- A specific element in an array is accessed by an index.
- All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

Array Based Implementation of List

List Structure

Index	Array	Position
List [0]		1
List [1]		2
List [2]		3
List [3]		4
List [4]		5

Array Name: List
List Size : 5
Start Position: 1
End Position: 5
Start Index: 0 i.e List[0]
End Index; 4 i.e List[4]
1st Element referred by: List[0]
5th Element referred by: List[4]
ith Element referred by: List[i-1]

Variable	Value
Max Size	5
Current Size	0

Max Size: Number of elements we can insert to the List.
Current Size: Number of elements already inserted to the List

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Is Empty(LIST)

- If (Current Size==0) "LIST is Empty"
- else "LIST is not Empty"

Is Full(LIST)

- If (Current Size=Max Size) "LIST is FULL"
- else "LIST is not FULL“

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Insert Element to End of the LIST

- Check whether the List is full or not.

✔ If List is full return error message ”List is full. Can’t Insert”.

✔ If List is not full.

❖ Get the position to insert the new element by

Position=Current Size+1

❖ Insert the element to the Position

❖ Increase the Current Size by 1 i.e. Current Size=Current Size+1

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Delete Element from End of the LIST

- Check that whether the List is empty or not
- If List is empty return error message ”List is Empty. Can't Delete”.
- If List is not Empty.

✔ Get the position of the element to delete by Position=Current Size

✔ Delete the element from the Position

✔ Decrease the Current Size by 1 i.e. Current Size=Current Size-1

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Insert Element to front of the LIST

- Check that weather the List is full or not
- If List is full return error message ”List is full. Can't Insert”.
- If List is not full.

✔ Free the 1st Position of the list by moving all the Element to one position forward i.e New Position=Current Position + 1.

✔ Insert the element to the 1st Position

✔ Increase the Current Size by 1 i.e. Current Size=Current Size+1

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Delete Element from front of the LIST

- Check that weather the List is empty or not
- If List is empty return error message ”List is Empty. Can't Delete”.
- If List is not Empty

✔ Move all the elements except one in the 1st position to one position backward i.e New Position = Current Position -1

✔ After the 1st step, element in the 1st position will be automatically deleted. Decrease the Current Size by 1.

✔ (i.e.) Current Size=Current Size-1

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Insert Element to nth Position of the LIST

- Check whether the List is full or not
- If List is full return error message ”List is full. Can't Insert”.
- If List is not full.

✔ If List is Empty, Insert element at Position 1.

✔ If (nth Position > Current Size)

✔ Return message “nth Position Not available in List” else

✔ Free the nth Position of the list by moving all Elements to one position forward except n-1,n-2, ... 1 Position (i.e) move only from n to current size position Elements. i.e New Position=Current Position + 1.

✔ Insert the element to the nth Position.

✔ Increase the Current Size by 1 i.e. Current Size=Current Size+1

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Delete Element from nth Position of the LIST

- Check that weather the List is Empty or not
- If List is Empty return error message ”List is Empty.”
- If List is not Empty.

✔ If (nth Position > Current Size)

✔ Return message “nth Position Not available in List”

✔ If (nth Position == Current Size)

✔ Delete the element from nth Position

✔ Decrease the Current Size by 1 i.e. Current Size=Current Size-1

✔ If (nth Position < Current Size)

✔ Move all the Elements to one position backward except n,n-1,n-2,... 1 Position i.e move only from n+1 to current size position Elements. i.e New Position=Current Position - 1.

✔ After the previous step, nth element will be deleted automatically.

✔ Decrease the Current Size by 1 i.e. Current Size=Current Size-1

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Search Element in the LIST

- Check that weather the list is empty or not.
- If List is empty, return error message “List is Empty”.
- If List is not Empty

✔ Find the Position where the last element available in the List by

✔ Last Position = Current Size

✔ For(Position 1 to Last Position)

✔ If(Element @ Position== Search Element) //If Element matches the search element

✔ return the Position by message “Search Element available in Position”

✔ Else return message “Search Element not available in the List”

Operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Print the Elements in the LIST

- Check that weather the list is empty or not.
- If List is empty, return error message “List is Empty”.
- If List is not Empty

✔ Find the Position where the last element available in the List by

✔ Last Position = Current Size

✔ For(Position 1 to Last Position)

✔ Print the Position and Element available at the position of List.

External References

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

1. https://nptel.ac.in/courses/106102064/
2. https://nptel.ac.in/courses/106103069/
3. https://nptel.ac.in/courses/106106127/
4. https://nptel.ac.in/courses/106106130/
5. https://www.geeksforgeeks.org/data-structures/
6. https://www.javatpoint.com/data-structure-tutorial
7. https://www.coursera.org/learn/data-structures
8. https://www.studytonight.com/data-structures/introduction-to-data-structures
9. https://www.youtube.com/watch?v=egb6qFw42JM (Tamil videos)


RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS
Thank You